

THE V(D)J RECOMBINATION MODEL AND FAST GENERATION PROBABILITIES IN VDJTOOLS

Mikhail Shugay^{1,2,3*}

¹Institute of Translational Medicine, Russian National Medical State University, Moscow, Russia

²Department of Genomics of Adaptive Immunity, Shemyakin–Ovchinnikov Institute of Bioorganic Chemistry
RAS, Moscow, Russia

³Institute of Molecular Biology NAS RA, Erevan, Armenia

Appendix M — vjtools model engine

Abstract

The vjtools model engine reimplements the Murugan–Marcou generative model of V(D)J recombination [1, 2] on the AIRR schema [3], storing every model factor as a tidy `polars` table so that the Bayesian-network structure is declared as data rather than baked into a file format. On top of this representation it computes the generation probability (P_{gen}) of a CDR3 — the total probability that the stochastic recombination process produces a given sequence — both for nucleotide queries (an exact scenario sum) and, more delicately, for amino-acid queries, where one must marginalize over all synonymous nucleotide realizations. For the amino-acid case we use the transfer-matrix factorization of [1, 4], but re-derive it from a single structural observation — *that the only coupling across a recombination-block boundary is codon translation, never the insertion Markov chain* — which yields a clean left/right partial-sum split (§M.4) that our native C++ core evaluates faster than OLGA (§M.5) while matching it to machine precision. The same tables and scenario enumeration drive sequence generation (ancestral sampling) and model inference (expectation–maximization), which we summarize in §M.7.

M.1. THE GENERATIVE RECOMBINATION MODEL

A rearranged receptor chain is the output of a stochastic process that (i) selects germline segments, (ii) trims their ends by exonuclease deletion, and (iii) inserts non-templated nucleotides at the junctions. Following [1, 2] we treat a *recombination scenario* r — the full list of hidden events — as a sample from a Bayesian network whose factors are the model parameters. For a VDJ locus (TRB, TRD, IGH) a scenario is

$$r = (V, J, D, \text{del}_V, \text{del}_J, \text{del}_D^{5'}, \text{del}_D^{3'}, \mathbf{m}^{VD}, \mathbf{m}^{DJ}), \quad (1)$$

with probability

$$P(r) = P(V)P(J)P(D|J)P(\text{del}_V|V)P(\text{del}_J|J)P(\text{del}_D^{5'}, \text{del}_D^{3'}|D)P_{\text{ins}}(\mathbf{m}^{VD})P_{\text{ins}}(\mathbf{m}^{DJ}). \quad (2)$$

*Correspondence: mikhail.shugay@gmail.com

A VJ locus (TRA, TRG, IGK, IGL) has no D segment; its scenario drops D , the two D -deletions and one insertion, and the gene factor becomes the joint $P(V)P(J | V)$ with a single VJ insertion. The observed CDR3 nucleotide sequence $\sigma(r)$ is the concatenation, from the conserved Cys 104 to the conserved Phe/Trp 118 inclusive,

$$\sigma(r) = \underbrace{\hat{V}}_{V \text{ germline}} \underbrace{\mathbf{m}^{VD}}_{N1} \underbrace{\hat{D}}_{D \text{ germline}} \underbrace{\mathbf{m}^{DJ}}_{N2} \underbrace{\hat{J}}_{J \text{ germline}}, \quad (3)$$

where $\hat{V}, \hat{D}, \hat{J}$ are the germline segments after the indicated deletions.

INSERTION CHAINS. Each non-templated segment $\mathbf{m} = m_1 \cdots m_\ell$ is drawn from a length distribution and a first-order Markov chain:

$$P_{\text{ins}}(\mathbf{m}) = P_{\text{ins}}(\ell) b(m_1) \prod_{i=2}^{\ell} R(m_i | m_{i-1}), \quad (4)$$

with b the first-nucleotide (stationary) bias and R the 4×4 dinucleotide transition matrix. Following IGoR/OLGA, the VD chain is read $5' \rightarrow 3'$ and the DJ chain $3' \rightarrow 5'$, so b_{DJ} biases the J -proximal insertion nucleotide.

GENERATION PROBABILITY. The quantity of interest is the probability of an *observed* CDR3, obtained by summing over every scenario that could have produced it,

$$P_{\text{gen}}(\sigma) = \sum_{r: \sigma(r)=\sigma} P(r), \quad P_{\text{gen}}(\alpha) = \sum_{\sigma: \tau(\sigma)=\alpha} P_{\text{gen}}(\sigma), \quad (5)$$

where τ is translation: an amino-acid query α additionally marginalizes over all synonymous nucleotide sequences. Both sums are finite but astronomically large if taken literally; § M.3 and § M.4 evaluate them by dynamic programming.

M.2. MODEL PARAMETERS AS DATA

`vdjtools` stores a model as a directory of long-format `polars` parquet tables plus a `manifest.json` that names each event and lists its conditioning parents — the edges of the Bayesian network of Eq. (2). Changing a locus’s conditioning (e.g. $P(D | J)$ versus a richer $P(D | V, J)$) is therefore a one-line manifest edit, not a format change, and a VJ locus is simply the same schema with the D -tables and second insertion absent. Each table holds the distribution of one event, normalized within its conditioning key by a single `polars group_by` — which is exactly the M-step of § M.7. Two conventions matter for the P_{gen} recursions.

PALINDROMIC DELETIONS. A negative deletion count encodes templated palindromic (P-) nucleotides. Each germline cut segment is pre-extended by up to `maxpal` reverse-complement nucleotides; a “deletion” of $-k$ means k P-nucleotides are retained. The stored count `del` is thus *biological* (negative = P-nucleotides), while the dense array index used at run time is `del + maxpal` ≥ 0 .

GERMLINE AS A SINGLE SOURCE OF TRUTH. All V/D/J germline sequences and the CDR3 anchor offsets resolve from `arda` by allele name; `arda`’s anchor convention is byte-identical to OLGA’s (a 0-based Cys 104 / [FW] 118 offset into the full germline), so no coordinate conversion is needed. A model never mixes germline sources: bootstrap models imported from OLGA keep OLGA germline to preserve exact- P_{gen} fidelity, whereas natively-inferred models use `arda`.

THE D-D EXTENSION. The schema is built to carry tandem- D ($n_D \in \{0, 1, 2\}$) rearrangements — a D -count event, a second D -gene table and a DD insertion — which OLGA/IGoR cannot represent. The bootstrap data used here (OLGA-drawn out-of-frame sequences) contains no tandem D by construction, so D - D is exercised only structurally until real data lands; the recursions below are written for $n_D \leq 1$ and degrade cleanly.

M.3. NUCLEOTIDE GENERATION PROBABILITY

For a nucleotide query σ the inner sum of Eq. (5) is a direct enumeration, because the germline segments and both junctions are pinned by σ . Writing $N = |\sigma|$ and letting a V (resp. J) contribute $\ell_V \geq 1$ (resp. $\ell_J \geq 1$) nucleotides — a valid rearrangement is never fully V - or J -deleted, matching OLGA —

$$P_{\text{gen}}(\sigma) = \sum_{V,J} P(V)P(J|V) \sum_{\substack{\ell_V, \ell_J \geq 1 \\ \sigma[0:\ell_V] = \hat{V}, \sigma[N-\ell_J:] = \hat{J}}} P(\text{del}_V)P(\text{del}_J) \Xi(\sigma[\ell_V:N-\ell_J]), \quad (6)$$

where for a VJ locus $\Xi(\mu) = P_{\text{ins}}(\mu)$ is the single-junction insertion weight of Eq. (4), and for a VDJ locus Ξ is itself a sum over the D gene, its two deletions and its placement,

$$\Xi(\mu) = \sum_D P(D|J) \sum_{\text{del}_D^{5'}, \text{del}_D^{3'}} P(\text{del}_D^{5'}, \text{del}_D^{3'}) \sum_{\substack{\text{placements } p \\ \mu[p:p+\ell_D] = \hat{D}}} P_{\text{ins}}(\mu[0:p]) P_{\text{ins}}(\mu[p+\ell_D:]). \quad (7)$$

Common-prefix / common-suffix matching restricts ℓ_V, ℓ_J to the genes that can actually flank σ , so Eq. (6) is fast in practice. This nucleotide sum is the quantity the E-step needs (§M.7); the native port reproduces OLGA's `compute_nt_CDR3_pgen` to machine precision.

M.4. AMINO-ACID GENERATION PROBABILITY BY TRANSFER MATRIX

The amino-acid case is harder: the codon marginalization in Eq. (5) reintroduces a sum over the unobserved junction nucleotides. Enumerating scenarios and summing synonymous codons position by position (a codon-constrained dynamic program of width $|\Sigma|$ over the nucleotide state) is correct but costs one $O(N)$ pass per $(D, \text{del}_D^{5'}, \text{del}_D^{3'}, \text{placement})$ skeleton — quadratic in N and prohibitive for long CDR3s. The transfer-matrix method removes the placement and insertion sums.

SETUP. Let $\alpha = \alpha_0 \dots \alpha_{L-1}$ be the query, $N = 3L$, and σ_i the (summed over) nucleotide at position i . Codon c imposes $\tau(\sigma_{3c}, \sigma_{3c+1}, \sigma_{3c+2}) = \alpha_c$. Because the codon and the insertion Markov chain each look back at most two and one nucleotide respectively, a left-to-right dynamic program with state $x_i = (\sigma_{i-1}, \sigma_{i-2})$ suffices: it can (a) close a codon when $i \equiv 2 \pmod{3}$ and (b) carry the Markov dependency. The width is $|\Sigma| = 25$ (four nucleotides plus a start sentinel, squared). This is the codon-marginalizing DP; the transfer matrix is what lets us reuse its partial results across all scenarios.

THE KEY STRUCTURAL FACT. In the block layout of Eq. (3), $\hat{V} - \mathbf{m}^{VD} - \hat{D} - \mathbf{m}^{DJ} - \hat{J}$, the blocks are coupled only through codon translation, never through an insertion Markov chain.

PROPOSITION 1 (Codon-only coupling). *Across every block boundary in Eq. (3) the recombination weight $P(r)$ factors into a left factor, the indicator(s) of the codon(s) straddling the boundary, and a right factor. No transition $R(\cdot | \cdot)$ spans a boundary.*

Sketch. The VD chain's leftmost nucleotide m_1^{VD} is drawn from the bias b_{VD} independent of the last V nucleotide, and its rightmost nucleotide carries no transition into $\hat{D}$; the D block is germline (fixed nucleotides, no chain); the DJ chain, read $3' \rightarrow 5'$, draws its J -proximal nucleotide from b_{DJ} independent of the first J nucleotide, and its D -proximal nucleotide carries no transition into $\hat{D}$. Hence every factor in Eq. (4) lives strictly inside one block. The only inter-block constraints are the codon indicators $\mathbb{1}[\tau(\cdots) = \alpha_c]$ for codons that straddle a boundary. $\square$ $\square$

Proposition 1 licenses computing the left and right halves of the sum *once each* and stitching them, instead of recomputing a full pass per scenario.

LEFT PARTIAL SUMS. Define, for each boundary position p (the position at which $\hat{D}$ starts) and each nucleotide state $x = (\sigma_{p-1}, \sigma_{p-2})$,

$$\Lambda_p(x) = \sum_{\substack{V, \text{del}_V, \ell_{VD} \\ \ell_V + \ell_{VD} = p}} P(V)P(\text{del}_V)P_{\text{ins}}(\ell_{VD}) [VD \text{ Markov}] \prod_{\text{codons} \subset [0, p)} \mathbb{1}[\tau = \alpha], \quad (8)$$

the total weight of all ways the V germline (with $\ell_V \geq 1$) plus the VD insertion fill $[0, p)$ ending in state x , with every codon that lies wholly in $[0, p)$ satisfied. Λ is built by one forward recursion: germline seeds are deposited at $p = \ell_V$, then the VD transfer step — multiply by b_{VD} on the first inserted nucleotide and R_{VD} thereafter, close codons at $p \equiv 2 \pmod{3}$, and add $P_{\text{ins}}(\ell_{VD})$ at each length — extends them, in $O(N|\Sigma|)$ total.

RIGHT PARTIAL SUMS. Symmetrically, for each D gene and boundary p (where $\hat{D}$ ends),

$$\Psi_p^D(y) = \sum_{J, \text{del}_J, \ell_{DJ}} P(D|J)P(J)P(\text{del}_J)P_{\text{ins}}(\ell_{DJ}) [DJ \text{ Markov}] \prod_{\text{codons} \subset [p, N)} \mathbb{1}[\tau = \alpha], \quad (9)$$

with $y = (\sigma_p, \sigma_{p+1})$. Because the DJ chain runs $3' \rightarrow 5'$, Ψ^D is built by a *backward* recursion from the J boundary — one per D gene, the $P(D|J)$ weight folded into the seed. There are at most n_D such passes.

STITCHING. Fixing D , its deletions (hence the fixed germline string $\hat{D}$ of length ℓ_D) and its start p_0 , thread the germline nucleotides of $\hat{D}$ through the DP from Λ_{p_0} to a state distribution at $p_1 = p_0 + \ell_D$ (closing any codons interior to $\hat{D}$, no Markov weight), then contract with $\Psi_{p_1}^D$. By Proposition 1 at most one codon straddles the D/DJ boundary p_1 ; the contraction is a 4×4 (or scalar, when $p_1 \equiv 0$) sum over that codon's free nucleotides. Summing over D , its deletions and p_0 ,

$$P_{\text{gen}}(\alpha) = \sum_D \sum_{\text{del}_D^{5'}, \text{del}_D^{3'}} P(\text{del}_D^{5'}, \text{del}_D^{3'}) \sum_{p_0} \left\langle \mathcal{T}_{\hat{D}} \Lambda_{p_0}, \Psi_{p_1}^D \right\rangle_{p_1}, \quad (10)$$

where $\mathcal{T}_{\hat{D}}$ is the germline-threading operator and $\langle \cdot, \cdot \rangle_{p_1}$ the codon-contracting stitch at p_1 . The insertion-length sums are already inside Λ and Ψ^D , so the outer work is $O(n_D \cdot \#\text{del}_D \cdot N \cdot \ell_D)$ with $\ell_D \ll N$ — versus the enumeration's extra factor of N from re-running the DP per placement. For a VJ locus there is no D and no split: a single codon-marginalizing DP over V -germline – VJ -insertion – J -germline templates suffices, which is already inexpensive.

REMARK 1. This is the $\Pi_L \cdot \Pi_R$ factorization of [1, 4] expressed on the `polars` tables. Where OLGA handles the D block with a sliding right-deletion recursion folded into Π_R , we keep Ψ^D purely J/DJ and slide a short explicit germline bridge (Eq. (10)); the two agree to machine precision (§M.5), and the explicit bridge is what generalizes to the tandem- D event.

M.5. CORRECTNESS AND BENCHMARK

The native transfer matrix is validated against two independent oracles: the transparent amino-acid scenario enumeration of §M.4 (pure Python), and OLGA itself [4]. On the human TRB model it reproduces the published worked example $P_{\text{gen}}(\text{CAWSVAPDRGGYTF} \mid \text{TRBV30}, \text{TRBJ1-2}) = 1.2036 \times 10^{-10}$ and agrees with OLGA to a maximum relative error of 1.2×10^{-15} over 25 marginalized (all V/J/D) and 6 V/J-restricted generated CDR3s — i.e. to machine precision. Table 1 reports timing, measured by `appendix/bench_pgen.py` on the human TRB OLGA default model, single-threaded, over 20 marginalized queries.

Table 1: Amino-acid P_{gen} throughput on human TRB (single thread, marginalized over all V/J/D). TM is the Eq. (10) transfer matrix in the native `_core`; OLGA is `compute_aa_CDR3_pgen`. Lower is better.

Method	ms / sequence	vs OLGA	max. rel. error vs OLGA
OLGA	5.3	$1.0 \times$	—
TM (native <code>_core</code>)	0.64	8.3 $\times$	1.2×10^{-15}

The pure-Python scenario enumeration of §M.4 is exact and serves as the correctness oracle, but is not a runtime path: unrestricted, it costs on the order of minutes per query (its extra factor of N and its lack of the left/right reuse), which is exactly the cost the transfer matrix removes.

M.6. DEGENERATE, AGNOSTIC AND HAMMING-BALL QUERIES

Three capabilities fall out of the same transfer matrix at essentially no extra cost. First, the codon test at each amino-acid position is a *membership test against an allowed-codon set* (a 64-bit mask), not an equality against one residue; a wildcard mask (any of the 20 amino acids) or any degenerate set is therefore evaluated in a single pass, giving motif/regex P_{gen} without enumerating the matching sequences. Second, conditioning on a V and/or J allele is a usage mask over the gene sums of Eqs. (8), (9); the empty restriction marginalizes over all functional genes — *V-agnostic* / *J-agnostic* P_{gen} .

Combining the two gives a fast one-mismatch query. The total P_{gen} of an amino-acid CDR3 and every sequence within Hamming distance 1 (one substitution) is, by inclusion–exclusion,

$$P_{\text{gen}}^{\leq 1}(\alpha) = \sum_{k=0}^{L-1} P_{\text{gen}}(\alpha_{k \rightarrow *}) - (L-1) P_{\text{gen}}(\alpha), \quad (11)$$

where $\alpha_{k \rightarrow *}$ wildcards position k . OLGA uses the same identity but pays a full (slow) transfer-matrix pass for each of the $L+1$ terms; here each term is one fast pass (Eq. (10)) with a single wildcard mask, so the native `_core` evaluates Eq. (11) at $8.7 \times$ OLGA’s throughput on human TRB (67.9 vs

7.8 ms/query) and $2.5\times$ on human TRA (25.1 vs 10.1 ms/query), matching it to a relative error of 10^{-15} throughout. A codon-boundary forward/backward sweep would collapse the $L+1$ passes into a single sweep for VJ loci; the D -placement sum couples positions in VDJ loci, so there the $L+1$ -pass identity is retained.

M.7. GENERATION AND INFERENCE ON THE SAME TABLES

GENERATION. Ancestral sampling walks the Bayesian network of Eq. (2) top-down: draw $(V, J[, D])$, the deletions, the insertion lengths, then the insertion nucleotides from the Markov chain, and assemble Eq. (3). Restricting to functional genes and enforcing $\ell_V, \ell_J \geq 1$ keeps every draw scoreable; sampled V/J-usage and CDR3-length distributions match the model marginals.

INFERENCE. Model parameters are estimated from out-of-frame reads by expectation–maximization. The E-step is exactly the scenario sum of §M.3: for each read it computes the posterior weight $P(r \mid \sigma) = P(r)/P_{\text{gen}}(\sigma)$ of every compatible scenario and accumulates soft counts into arrays laid out like the model tables; optionally the enumeration is restricted to the genes `arda` aligns (an *aligned-gene mask*), which prunes the V/D/J sums without changing the fixed point. The M-step renormalizes each event table within its conditioning key — a single `polars group_by` — reproducing the manifest’s Bayesian network. On OLGA-drawn data this is a closed-loop oracle: EM must recover the generating marginals, giving a self-checking target before any real data exists. The E-step inner loop is the second native hot path (the first being P_{gen}); with the aligned-gene mask it makes VDJ inference practical.

REFERENCES

- [1] Anand Murugan, Thierry Mora, Aleksandra M. Walczak, and Curtis G. Callan. Statistical inference of the generation probability of T-cell receptors from sequence repertoires. *Proceedings of the National Academy of Sciences*, 109(40):16161–16166, 2012.
- [2] Quentin Marcou, Thierry Mora, and Aleksandra M. Walczak. High-throughput immune repertoire analysis with IGoR. *Nature Communications*, 9(1):561, 2018.
- [3] Jason A. Vander Heiden, Susanna Marquez, Nishanth Marthandan, et al. AIRR community standardized representations for annotated immune repertoires. *Frontiers in Immunology*, 9:2206, 2018.
- [4] Zachary Sethna, Yuval Elhanati, Curtis G. Callan, Aleksandra M. Walczak, and Thierry Mora. OLGA: fast computation of generation probabilities of B- and T-cell receptor amino acid sequences and motifs. *Bioinformatics*, 35(17):2974–2981, 2019.